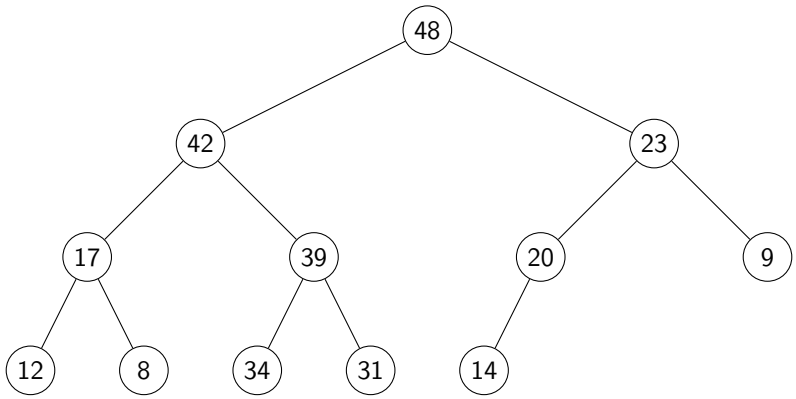
Motivation

Abstract Data Type **Priority Queue** is a collection of integers with the following operations.

`D.Init()`	Initializes the priority queue
`D.Insert(`x`)`	Inserts x into the Priority Queue.
int `D.ExtractMax()`	Extract the maximum element from the Priority Queue.

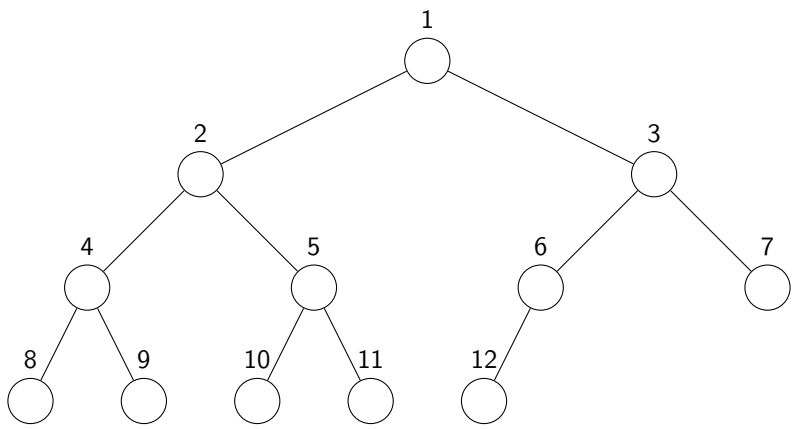


Definition 1. A **max-heap** is a nearly complete binary tree that satisfies the heap property. The max-heap property is that for any node x, if c_l and c_r are the left and right children of x, then $x.\texttt{key} \geq c_l.\texttt{key}$ and $x.\texttt{key} \geq c_r.\texttt{key}$.

Definition 2. A **min-heap** is a nearly complete binary tree that satisfies the heap property. The min-heap property is that for any node x, if c_l and c_r are the left and right children of x, then $x.\texttt{key} \leq c_l.\texttt{key}$ and $x.\texttt{key} \leq c_r.\texttt{key}$.

Introduction and Methods

Nearly complete binary tree.

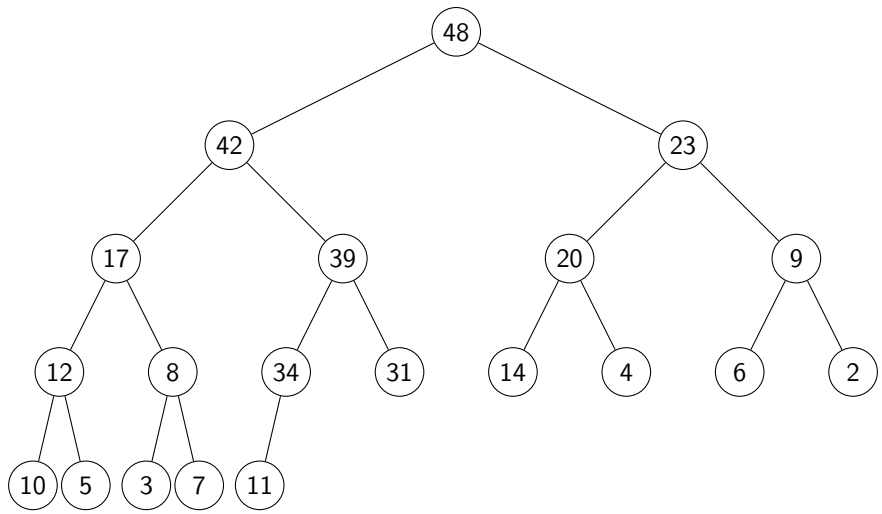


Only nodes in the bottom right of the tree are missing.

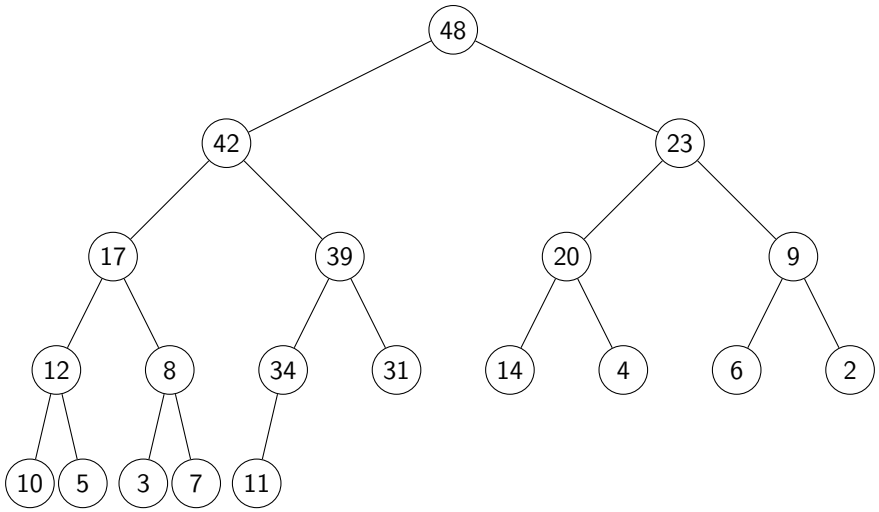
```
function Parent(i)
    return ⌊i/2⌋
end function
```

```
function Left(i)
    return 2i
end function
```

```
function Right(i)
    return 2i + 1
end function
```



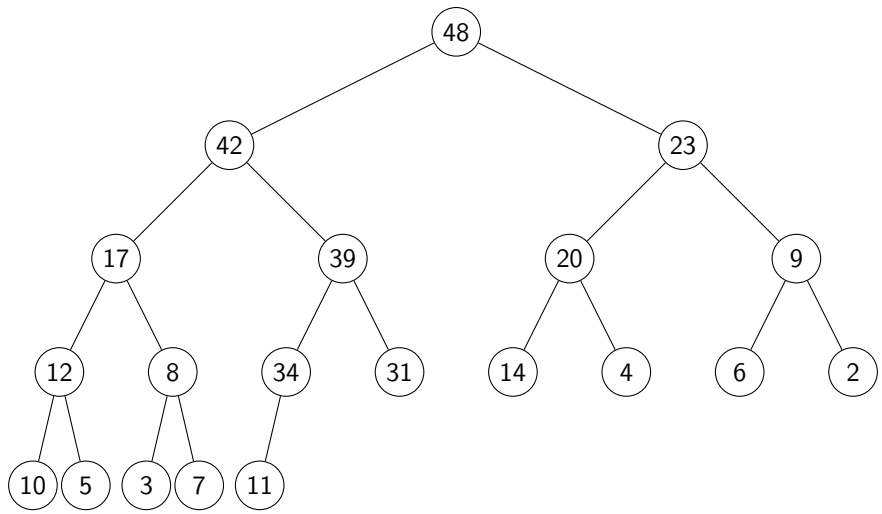

```
function MaxHeapInsert(A[],size,K)
    size = size + 1
    i = size
    A[i] = K
    while (i > 1) and (A[Parent(i)] < A[i]) do
        Swap(A[i],A[Parent(i)])
        i = Parent(i)
    end while
end function
```




```
function HeapExtractMax(A[],size)
    if size < 1 then
        error("heap underflow")
    end if
    maxKey = A[1]
    A[1] = A[size]
    size = size − 1
    MaxHeapify(A[],size)
    return maxKey
end function
```

```
function MaxHeapify(A[],size)
    flagDone = false
    j = 1
    repeat
        L = Left(j)
        R = Right(j)
        ▷ Find index of largest element between A[j], A[L], A[R]
        largest = j
        if (L ≤ size) and A[L] > A[largest] then
            largest = L
        end if
        if (R ≤ size) and A[R] > A[largest] then
            largest = R
        end if
        if (j ≠ largest) then
            Swap(A[j],A[largest])
        else
            flagDone = true
        end if
        j = largest
    until flagDone
end function
```

```
function MaxHeapify(A[],size)
    flagDone = false
    j = 1
    repeat
        L = Left(j)
        R = Right(j)
        ▷ Find largest element of A[j], A[L], A[R]
        largest = arg max_{{j,L,R}} {A}
        if (j ≠ largest) then
            Swap(A[j],A[largest])
        else
            flagDone = true
        end if
        j = largest
    until flagDone
end function
```




```
function MaxHeapIncreaseKey(A[],size,i,K)
    if K < A[i] then
        error("new key is less than current key")
    end if
    A[i] = K
    while (i > 1) and (A[Parent(i)] < A[i]) do
        Swap(A[i],A[Parent(i)])
        i = Parent(i)
    end while
end function
```

Input: Array B of n elements

Output: A permutation of B such that $B[1] \leq B[2] \leq \cdots \leq B[n]$

```
function HeapSort(B[],n)
    size = 0
    for i = 1 to n do
        MaxHeapInsert(A,size,B[i])
    end for
    for i = n downto 1 do
        B[i] = HeapExtractMax(A, size)
    end for
end function
```

Alternative Priority Queue Implementations

Operations:

`D.Init()`	Initializes the priority queue
`D.Insert(`x`)`	Inserts x into the Priority Queue.
int `D.ExtractMax()`	Extract the maximum element from the Priority Queue.

Heap:

`D.Init()`	$\Theta(1)$ time
`D.Insert(`x`)`	$\Theta(\log(s))$ time
int `D.ExtractMax()`	$\Theta(\log(s))$ time

where s is the number of elements in the data structure.

Array implementation

```
function D.Init()
    length = 0
end function
```

```
function D.Insert(x)
    length = length + 1
    A[length] = x
end function
```

```
function D.ExtractMax()
    if length < 1 then
        error("array underflow")
    end if
    for i = 1 to length − 1 do
        if A[i] > A[length] then
            Swap(A[i],A[length])
        end if
    end for
    x = A[length]
    length = length − 1
    return x
end function
```

Sorted array implementation

```
function D.Init()
    length = 0
end function
```

```
function D.Insert(x)
    length = length + 1
    A[length] = x
    k = length
    while (k > 1) and (A[k − 1] > A[k]) do
        Swap(A[k − 1], A[k])
        k = k − 1
    end while
end function
```

```
function D.ExtractMax()
    if length < 1 then
        error("array underflow")
    end if
    x = A[length]
    length = length − 1
    return x
end function
```

Priority queue sort

Input: Array A of n elements

Output: A permutation of A such that $A[1] \leq A[2] \leq \cdots \leq A[n]$

```
function Sort(A[],n)
    D.Init()
    for i = 1 to n do
        D.Insert(A[i])
    end for
    for i = n downto 1 do
        A[i] = D.ExtractMax()
    end for
end function
```

Operation	Heap	Array	Sorted Array
`Insert`	$\log(s)$	c	s
`ExtractMax`	$\log(s)$	s	c
`Sort`	$n\log(n)$	n^2	n^2

$s =$ number of elements in Priority Queue

$n =$number of elements in array being sorted.

Examples

Example 1. Determine the asymptotic complexity of the given algorithm:

```
function Func(A[],n)
    D.Init()
    for i = 1 to n do
        for j = 1 to n do
            D.Insert(A[i] × A[j])
        end for
    end for
    x = 0
    for i = 1 to n do
        x = x + D.ExtractMax()
    end for
    return x
end function
```

`D.Insert`(x) is in $\Theta(s)$, where s is the number of elements in D

`D.ExtractMax`() is in $\Theta(1)$

Example 2. Determine the asymptotic complexity of the given algorithm:

```
function Func(A[],n)
    D.Init()
    for i = 1 to n do
        for j = 1 to n do
            D.Insert(A[i] × A[j])
        end for
    end for
    x = 0
    for i = 1 to n do
        x = x + D.ExtractMax()
    end for
    return x
end function
```

`D.Insert`(x) is in $\Theta(1)$

`D.ExtractMax`() is in $\Theta(s)$, where s is the number of elements in D

Example 3. Determine the asymptotic complexity of the given algorithm:

```
function Func(A[],n)
    D.Init()
    for i = 1 to n do
        for j = 1 to n do
            D.Insert(A[i] × A[j])
        end for
    end for
    x = 0
    for i = 1 to n do
        x = x + D.ExtractMax()
    end for
    return x
end function
```

`D.Insert`(x) is in $\Theta(\log(s))$, where s is the number of elements in D

`D.ExtractMax`() is in $\Theta(\log(s))$, where s is the number of elements in D